

How Polynexus HMS Works

The complete procedure, SaaS and on-premise

October 2026 · For the Polynexus team

THE JOURNEY AT A GLANCE

Part	What it covers
1. The product	What hospitals get, and the two ways we deliver it
2. What runs where	The services, and where data lives
3. People and permissions	Polynexus staff, hospital staff
4. SaaS: the full procedure	From a new hospital to monthly billing
5. On-premise: the full procedure	From build to a licensed, backed-up server
6. Licences and their safeguards	How no unauthorised licence can be sold
7. Changes during a contract	More modules, more users, renewals, cancellations
8. Updates	Shipping a new version in each mode
9. Routine checks	What to look at daily, weekly, monthly
10. Where to find more	The other guides

Before you start

1

The product

Polynexus HMS is one system with four parts that a hospital can buy separately:

Part	What it does
HMS	Patients, OPD, IPD, nursing, beds, billing, emergency, OT, ICU, lab, radiology, pharmacy dispensing, the 3D anatomy viewer
CRM	Enquiries, call queue and callbacks, referrals, packages, feedback, WhatsApp and IVR
ERP	Inventory, pharmacy stores, finance and accounts, HR, support services
MIS	Reports, analytics, NABH quality indicators

The same code runs in two modes. Nothing is built twice; the mode decides how the hospital is served and how its access is controlled.

	SaaS (off-premise)	On-premise
Where it runs	Our servers	The hospital's own server (Linux or Windows)
Hospitals per server	Many, each kept separate	One (or a few branches of one group)
Who controls access	Our subscription records, live	A signed licence file on their server
Internet needed	Yes	No — works fully offline
How they pay	Monthly or yearly invoices from the console	Licence fee per term; renewal = new licence
Updates	We update everyone at once	We send a new bundle; their IT runs the upgrade
Code on their side	None	Compiled program only — no source code

2

What runs where

Both modes run the same set of services, in Docker containers:

Service	Job
nginx	Serves the web app and passes <code>/api</code> to the backend; HTTPS
web	The backend (Django) — all business rules, permissions, licence checks
worker	Background jobs: messages, reports, imports
beat	Scheduled jobs: hourly licence check, reminders, daily tasks
postgres	The database
redis	Job queue and cache
backup	Nightly database and documents backup, 30 days kept (on-premise)

Data. Patient data sits in PostgreSQL; sensitive fields are encrypted with keys from the server's `.env` file. In SaaS that's our server, with each hospital's records separated by hospital. On-premise it's the hospital's server, and nothing leaves it unless they turn on an integration (WhatsApp, IVR, ABDM) that their licence includes.

The `.env` file holds the server's secrets and data keys. Losing it means losing access to encrypted data, so it is backed up separately from the database, in both modes.

3

People and permissions

Polynexus staff use the SaaS console. Each has a role, and a permanent staff code (for example `PNX-0007`) that is never reused.

Role	Can
SaaS Owner	Everything: tenants, billing, security review, staff, and signing licences
Platform Admin	Onboard and manage hospitals, modules, support — not licences, unless an Owner allows it
Billing	Subscriptions and invoices
Support (L1, L2, Lead)	Support tickets; L2 and Lead can enter a hospital to help
Customer Success	View hospitals, support
Security Auditor	Security events and audit trails
DevOps	Platform health

Licence rights sit on top of the role: **SaaS Owners** sign licences; any other staff member can only *request* them, and only after an Owner switches on *Allow licences* for them. Every licence step needs a fresh 2FA code.

Hospital staff sign in to their own hospital only. The hospital's administrator creates users and gives each a role (doctor, nurse, receptionist, pharmacist, accountant, and so on). Each role sees only its own menus and actions, inside the modules the hospital has bought. Every view and change of a patient record is written to the patient audit trail.

Blocking someone (in either the console or a hospital) ends their sessions on every device immediately — not at the next login.

4

SaaS: the full procedure

- 1. Onboard the hospital.** SaaS console → + *Onboard Hospital*: name, address, administrator email, plan (tier and billing cycle), and the modules they bought. This creates the hospital, its administrator account and its subscription.
- 2. The hospital signs in.** The administrator receives their login, sets a password (and 2FA), creates their staff accounts and roles, and starts work. Only the modules they bought appear in their menus.
- 3. Billing.** Billing staff record an invoice for each billing period in the console (Invoices); they are not created automatically. When payment arrives, mark the invoice paid — the licence register uses paid invoices to flag licences issued without payment. The hospital sees its plan, modules, next billing date and invoices (with PDF downloads) in **Settings** → **Subscription**.
- 4. Support.** Hospitals raise tickets; support staff answer them in the console. Support L2 and Lead can switch into a hospital to see what the user sees.
- 5. Watching usage.** Analytics and usage snapshots in the console show active users, patients and module use per hospital.
- 6. Suspending or ending.** Suspend a hospital that hasn't paid (deactivate it in Tenants & Subscriptions): its users' requests are refused until it's reactivated, and nothing is deleted. Reactivating restores everything as it was.

5

On-premise: the full procedure

Each step names who does it. The detailed commands are in the Client Delivery guide.

1. Build the bundle (Polynexus, each release). On our build PC: pull both repositories, fetch the 3D model (`git lfs pull`), download the revocation list from the console into `Backend/apps/licensing/revocations.lic` , then `make bundle VERSION=x.y.z` . The bundle holds the compiled backend, the obfuscated frontend, PostgreSQL and Redis images, and the installers. **One bundle serves every client.**

2. Deliver (Polynexus). Send `bundle-x.y.z.tar.gz` with its SHA-256 checksum, by USB or a download link. It contains no source code and no secrets.

3. First install run (hospital IT). Unpack on the server and run `install.sh` (Linux) or `install.ps1` (Windows). It loads the images without internet, creates `.env` with fresh secrets and a **deployment ID**, reads the **machine fingerprint**, writes both into `config/REQUEST-LICENCE.txt` , and stops. They send us that file.

4. Licence (Polynexus, two people).

1. A member of the licence team requests it in the SaaS console (hospital → License), with the deployment ID, fingerprint, expiry, user and bed limits, features, and their 2FA code.
2. A SaaS Owner checks the terms and that the hospital has paid, and approves with their 2FA code. The licence is signed on the SaaS server and downloads as `LIC-YYYY-XXXXXXX.lic` .
3. Check it with `python -m license_issuer verify <file>` and send it to the hospital. It's safe to email: it can't be edited and works only on their server.

5. Second install run (hospital IT). Save the file as `config/license.lic` and run the installer again. It prepares the database, asks for the administrator email and password, activates the licence, creates the hospital and starts everything. Open the address it prints.

6. Domain and HTTPS (hospital IT). Point a name such as `hms.hospital.com` at the server in their DNS, then run `configure-domain.sh` / `.ps1` : their own certificate, Let's Encrypt, or plain HTTP for internal use only.

7. Backups (hospital IT). Every night at 02:00 the database and documents are backed up, 30 days kept, result in `backups/LAST-BACKUP.txt` . Set `BACKUP_DIR` to a NAS or external drive so backups leave the server, keep a copy of `.env` separately, and practise one restore.

8. Daily running. The licence is checked when the server starts, on every request (cached 10 minutes) and hourly. Administrators see a yellow banner 30 days before expiry, a red one during the grace period, and after that the system becomes **read-only**: everything can be viewed and printed, nothing changed. Data is never deleted because of a licence.

6

Licences and their safeguards

A licence is a small file signed with our private key. The server checks it offline with the public key built into the program, so any edit breaks it. It states: the hospital, deployment ID, start and expiry, grace period, user and bed limits, features, the machine it's bound to (optional), and the staff codes of who requested and who approved it.

These safeguards make sure no licence reaches a hospital without the company knowing:

#	Safeguard	What it stops
1	Blocking a user ends their sessions instantly	A departing employee using a still-open session
2	Only Owners sign; others need an Owner's permission to even request	Platform Admins or other staff issuing licences on their own
3	A fresh 2FA code for every licence step	A stolen password or an unlocked laptop being enough
4	Two-person rule: request, then Owner approval	One person issuing a licence alone
5	Staff codes signed into every licence, shown in the register, verify , and the hospital's Settings → License	Anyone hiding who issued a licence
6	Owners emailed for every request and licence; register flags <i>No paid invoice</i>	Licences issued without payment going unnoticed
7	Private key only on the SaaS server; the command-line tool only verifies	Licences signed on a laptop, outside the console
8	Revocation list built into each release	A revoked licence working forever on an offline server
9	Usage reports naming an unknown licence raise a critical alert	A licence signed outside the console going undetected

When a staff member leaves: an Owner blocks them (Licence team → Block, with 2FA). Their sessions end at once, their licence right is removed, and their pending requests are cancelled. Filter the register by their code to review every licence they issued, and revoke any that shouldn't exist.

If you suspect the signing key has leaked (for example an unknown-licence alert): stop issuing, tell the Owners, and plan a key change with the development team. A new key means re-issuing every licence, so it is a planned operation, not a quick fix.

7

Changes during a contract

Change	SaaS	On-premise
Add modules	Console → the hospital's modules. Takes effect at once; adjust the subscription and invoice	Issue a new licence with the extra features (request + approval); the hospital uploads it in Settings → License, no restart
More users or beds	Update the subscription	New licence with higher limits
Renewal	Next invoice; nothing to install	Ask for a usage report first, then a new licence with the new expiry. Same deployment ID and fingerprint
New server hardware	Not applicable	Their new fingerprint → new licence
Non-payment	Suspend the hospital	Don't renew; it becomes read-only after the grace period
Contract cancelled	Suspend, export their data if asked	Revoke the licence; it stops on their next upgrade, or at expiry

Usage reports (on-premise). In Settings → License, the hospital downloads a usage report (user, bed and patient counts — no patient details) and sends it with their renewal. Upload it in the console under On-Premise Licences: it shows their real usage next to the licence limits. A report that was edited is flagged as unreliable; one for a licence we never issued is refused and raises an alert.

8

Updates

SaaS. Build the SaaS images, deploy them to our server, run migrations. Every hospital is on the new version at once.

On-premise.

1. Download the revocation list into the build, then `make bundle` with a higher version number.
2. Send the bundle to each hospital.
3. Their IT unpacks it into a **new folder** and runs `upgrade.sh /opt/hms` or `upgrade.ps1 -InstallDir C:\hms`.
The upgrade backs up the database first, keeps `.env`, the licence and all data, migrates and restarts.

9

Routine checks

When	Who	What
Daily	SaaS Owner	Licence requests waiting for approval; licence emails
Weekly	SaaS Owner	Register: any <i>No paid invoice</i> flags; licences expiring in 60 days
Weekly	Hospital IT (on-premise)	<code>backups/LAST-BACKUP.txt</code> says OK
Monthly	Billing	Unpaid invoices; suspend or chase
Each release	Whoever builds	Fresh revocation list in the build
When staff join or leave	SaaS Owner	Licence team: grant or remove the right; block leavers the same day
Each renewal	Licence team	Usage report uploaded and checked against the limits

10

Where to find more

Guide	For
docs/CLIENT_DELIVERY.pdf	Step-by-step delivery to a new on-premise client, with every command
deploy/RUNBOOK.md	Operations reference: installing, licences, upgrades, troubleshooting, domain, backups and restore
docs/TRIAL_INSTALL_WINDOWS.pdf	Practising a complete install on a Windows PC
docs/CHANGES_2026-10-10.pdf	What changed on 10 October 2026, and how to test it